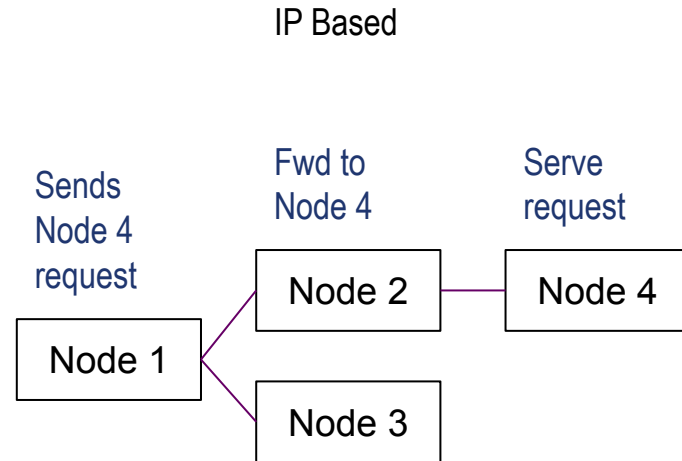
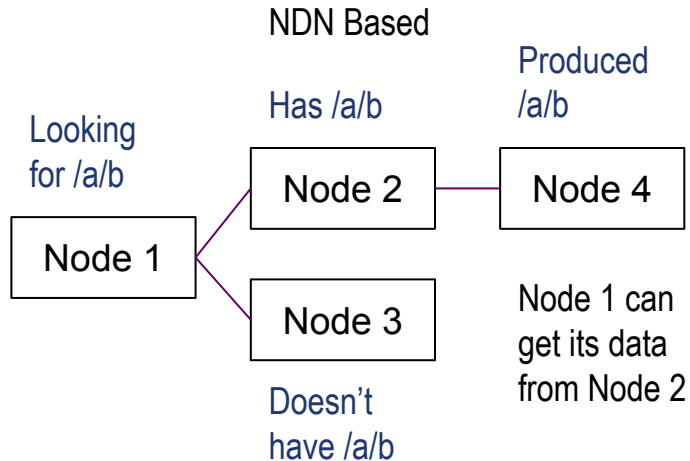


FPGA-Based Acceleration of the Forwarder



Background

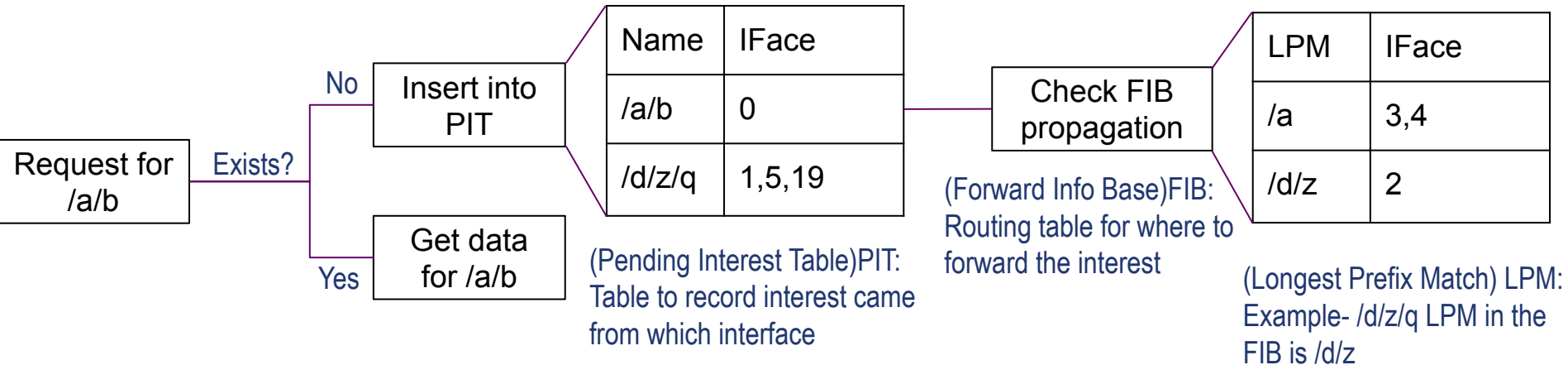
- **NDN uses hierarchical names for retrieval of content**
 - Any node can now serve the content if it has it
 - IP based requires the specified destination handle the request



Background

- **Forwarding logic**

- **Determines if data resides on node or needs to be forwarded based on hierarchical name**



Representing Names

- **Hierarchical names are variable in length**
 - **Unlike IP which are 32(v4) or 128(v6) bits**
 - **Need to transform into unique/fixed-length identifiers**
 - **Each prefix should be unique**
 - **Use a hashing algorithm (in this case, SIP Hash)**

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed
`/aaaaa/bbbbbb/ccccc/dddddd`
Do a 6 round computation

Total rounds so far: 6

Hashes produced so far: 1

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed
`/aaaaa/bb`bbb/cccc/dddddd
Do a 2 round computation
Total rounds so far: 8
Hashes produced so far: 1

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed
/aaaaa/bbbbb/ccccc/ddddd
Do a 6 round computation
Total rounds so far: 14
Hashes produced so far: 2

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed

/aaaaa/bbbbbb/ccccc/dddddd

Do a 2 round computation

Total rounds so far: 16

Hashes produced so far: 2

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed
/aaaaa/bbbbbb/ccccc/dddddd
Do a 6 round computation
Total rounds so far: 22
Hashes produced so far: 3

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed

/aaaaa/bbbbbb/ccccc/ddddd

Do a 2 round computation

Total rounds so far: 24

Hashes produced so far: 3

SIP Hash Overview and Challenge

- The SIP Hash algorithm for NDN
 - Every 8 bytes goes through 2 rounds of XOR|ADD|ROTATE
 - Every end of prefix goes through 6 rounds of XOR|ADD|ROTATE and produces a 64 bit hash
 - More prefixes in the name, the more computation needed
- /aaaaa/bbbbbb/ccccc/dddddd

Do a 6 round computation

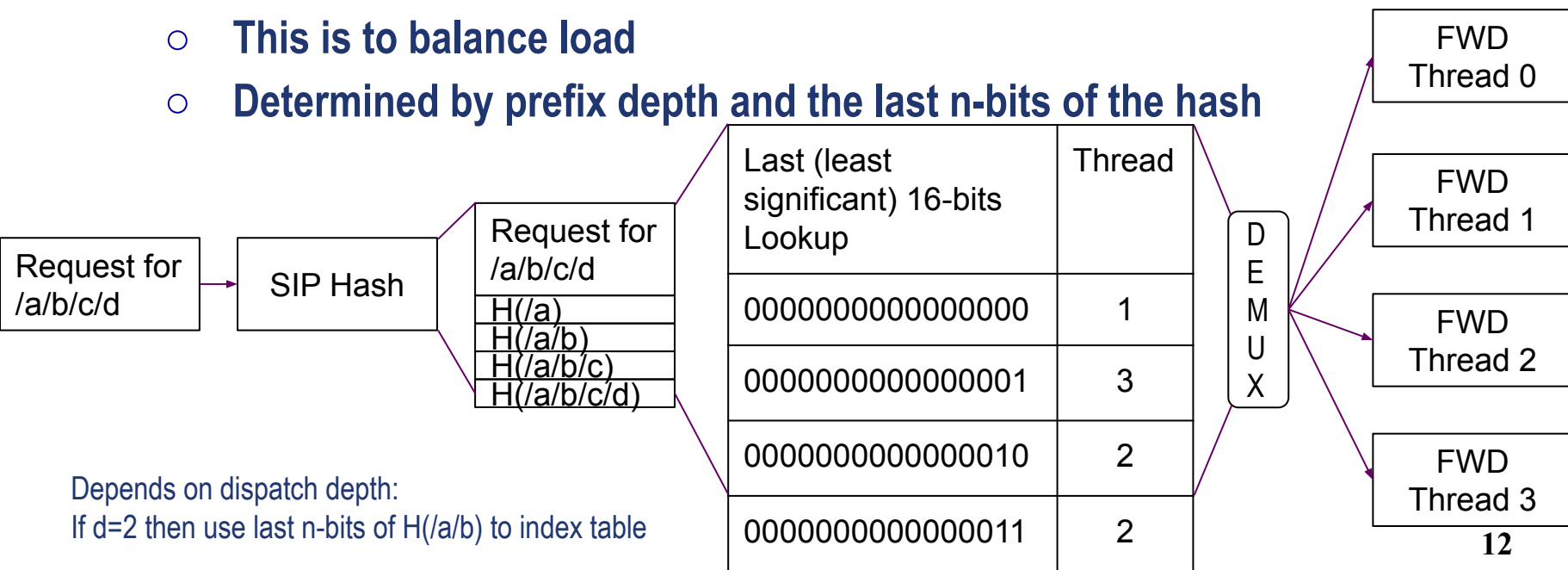
Total rounds so far: 30

Hashes produced so far: 4

30 rounds of computation for a
name of 25 characters!

Name Dispatch Table

- Name dispatch table used for multi-threaded forwarders
 - This is to balance load
 - Determined by prefix depth and the last n-bits of the hash



Challenge

- **Size of dispatch tables can be an issue for multi-threaded forwarders**
 - **Requires fine balancing of table size**
 - Large tables can lead to poor locality during lookup
 - Small tables can lead to imbalanced thread workload

H(/a/b):

```
10000100000011110_00101010000000111_0101111000111100_0001111111100001
10000100000011110_00101010000000111_0101111000111101_0001111111100001
10000100000011110_00101010000000111_0101111000111110_0001111111100001
10000100000011110_00101010000000111_0101111000111111_0001111111100001
```

If last 16 bits, sent to 1 thread

Challenge

- **Size of dispatch tables can be an issue for multi-threaded forwarders**
 - **Requires fine balancing of table size**
 - Large tables can lead to poor locality during lookup
 - Small tables can lead to imbalanced thread workload

H(/a/b):

```
10000100000011110_00101010000000111_0101111000111100_0001111111100001
10000100000011110_00101010000000111_0101111000111101_0001111111100001
10000100000011110_00101010000000111_0101111000111110_0001111111100001
10000100000011110_00101010000000111_0101111000111111_0001111111100001
```

If last 17 bits, sent to 2 different threads

Challenge

- **Size of dispatch tables can be an issue for multi-threaded forwarders**
 - **Requires fine balancing of table size**
 - Large tables can lead to poor locality during lookup
 - Small tables can lead to imbalanced thread workload

H(/a/b):

```
10000100000011110_00101010000000111_0101111000111100_0001111111100001
10000100000011110_00101010000000111_0101111000111101_0001111111100001
10000100000011110_00101010000000111_0101111000111110_0001111111100001
10000100000011110_00101010000000111_0101111000111111_0001111111100001
```

If last 18 bits, sent to 4 different threads

Design Exploration

- Our goal is to overlap any computation as much as possible
 - SIP Hash has some computational overlaps
 - Dispatch lookup and hashing can mostly be done in parallel
- Not feasible on CPU to use threads for relatively short tasks that have overlaps
 - This is where FPGA has the advantage

Design Exploration SIP Hash

- Overlap hash computations
 - The 6 rounds needed at each end of prefix does not affect later prefixes

/aaaaa/bbbbb



Design Exploration SIP Hash

- Overlap hash computations
 - The 6 rounds needed at each end of prefix does not affect later prefixes

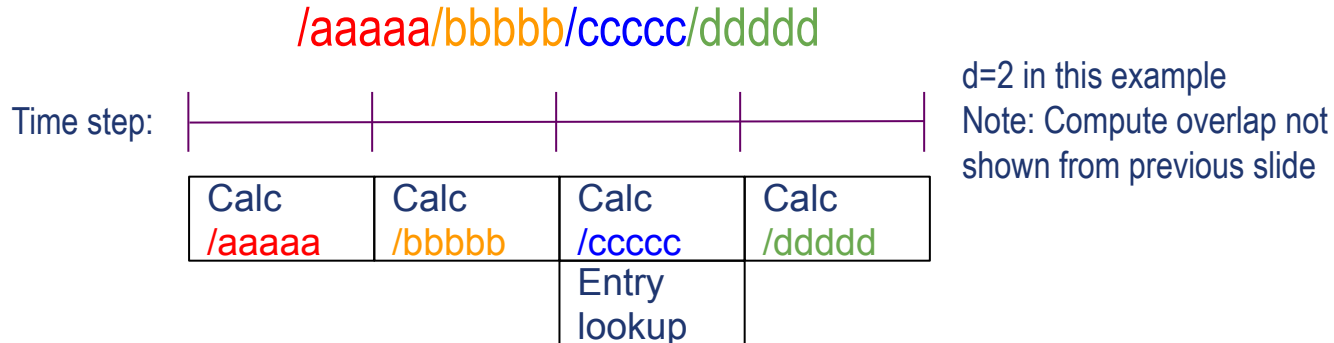
/aaaaa/bbbbb

While **/aaaaa** is computing its 6 rounds,
The design will also start to process **/bbbbbb** in parallel

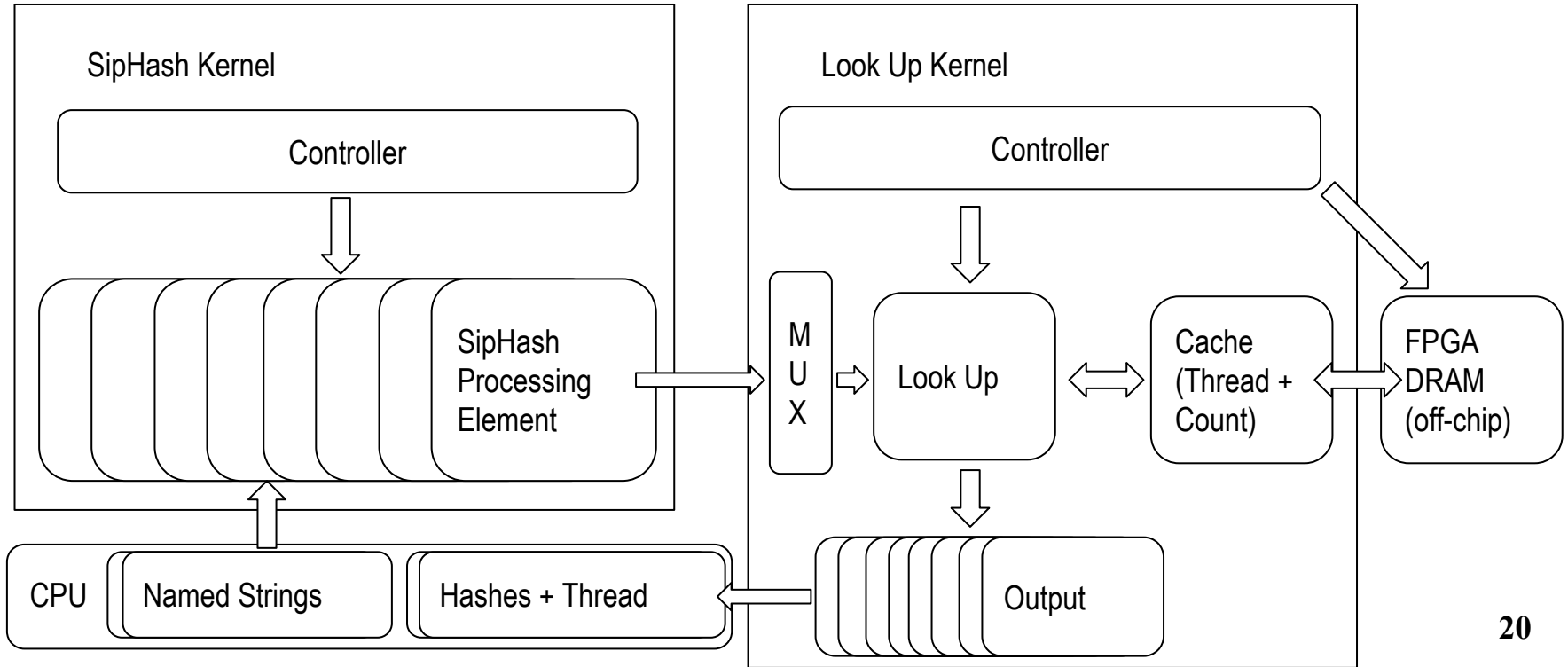


Design Exploration Dispatch Table

- Overlap lookup with hash computations
 - Once hash of specified depth is computed, send to lookup table while rest of hash computation proceeds in parallel
 - No benefit if depth is greater than number of prefixes of the name
- Caching for frequent entry hits



Design Exploration



Testing Results

- **Stand alone comparison methodology**
 - Compared against program that only computes hashes and table lookups
 - Comparison is against the same amount of threads and processing elements
 - I.E. 2 thread vs 2 PE, 8 thread vs 8 PE
 - 1 table look up thread since the DPDK uses 1 thread for look up
 - CPU Specs:
 - Xeon Gold 6244 @ 3.6 GHz
 - L1 Cache: 32K
 - L2 Cache: 1024K
 - L3 Cache: 25344K
 - A dump of my home folder
 - Average 64 characters per prefix
 - Average # of prefixes is at ~10
 - Common # of prefixes is around 8
 - 1,209,841 entries 17,743,189 prefixes

Per Name - 1 Thread

Depth Bits	16	18	20	22	24	CPU In Nano sec
2	1004.78	996.06	994.53	931.85	993.98	
4	946.60	956.27	919.16	941.34	991.62	
8	982.80	978.64	973.65	1020.64	1023.17	
16	1004.90	1054.91	1006.02	1138.48	1183.87	
32	1015.38	1071.31	1100.20	1097.63	1123.52	
2	343.51	344.14	343.92	345.02	343.53	FPGA In Nano sec
4	346.61	349.34	344.15	343.88	343.57	
8	344.79	351.50	343.59	344.50	342.97	
16	345.98	344.92	342.98	347.20	342.97	
32	345.29	344.06	345.28	342.17	342.37	

Speed Up Factor Over CPU				
2.93	2.89	2.89	2.70	2.89
2.73	2.74	2.67	2.74	2.89
2.85	2.78	2.83	2.96	2.98
2.90	3.06	2.93	3.28	3.45
2.94	3.11	3.19	3.21	3.28

- Depth: ex. /a/b/c, /a=1, /a/b=2, /a/b/c=3
- Bits: LSBs required for lookup
 - ex. 2^{16} entries need last 16 bits:
 ...01001011111011110011

Per Name - 2 Threads

Depth Bits	16	18	20	22	24	
2	648.07	629.35	665.05	655.86	631.31	CPU In Nano sec
4	628.03	665.08	640.86	602.73	622.75	
8	653.43	658.41	584.97	720.01	651.44	
16	667.93	684.52	737.58	771.29	827.07	
32	670.70	731.73	771.49	784.35	785.77	
2	179.05	177.89	175.32	176.23	175.88	FPGA In Nano sec
4	178.89	176.94	177.10	176.56	175.61	
8	178.75	178.93	177.00	176.00	175.21	
16	180.73	178.43	178.37	177.11	175.26	
32	182.14	178.85	175.80	178.16	176.58	

Speed Up Factor Over CPU				
3.62	3.54	3.79	3.72	3.59
3.51	3.76	3.62	3.41	3.55
3.66	3.68	3.30	4.09	3.72
3.70	3.84	4.13	4.35	4.72
3.68	4.09	4.39	4.40	4.45

- Depth: ex. /a/b/c, /a=1, /a/b=2, /a/b/c=3
- Bits: LSBs required for lookup
 - ex. 2^{16} entries need last 16 bits:
 ...01001011111011110011

Per Name - 4 Threads

Depth Bits	16	18	20	22	24	CPU In Nano sec
2	340.23	373.56	340.85	355.28	363.34	
4	362.80	333.56	361.80	338.34	373.35	
8	378.11	366.55	393.13	407.37	412.30	
16	417.86	448.05	458.54	503.99	490.16	
32	391.84	435.72	499.84	483.77	503.91	
2	104.64	95.35	96.75	95.39	94.46	FPGA In Nano sec
4	103.94	99.63	101.64	97.38	99.52	
8	95.79	93.25	98.24	95.12	95.60	
16	99.28	100.24	100.63	95.68	99.24	
32	98.35	96.78	96.04	94.92	94.90	

Speed Up Factor Over CPU				
3.25	3.92	3.52	3.72	3.85
3.49	3.35	3.56	3.47	3.75
3.95	3.93	4.00	4.28	4.31
4.21	4.47	4.56	5.27	4.94
3.98	4.50	5.20	5.10	5.31

- Depth: ex. /a/b/c, /a=1, /a/b=2, /a/b/c=3
- Bits: LSBs required for lookup
 - ex. 2^{16} entries need last 16 bits:
 ...01001011111011110011

Per Name - 8 Threads

Depth Bits	16	18	20	22	24	CPU In Nano sec
2	217.54	201.26	216.41	206.42	218.53	
4	218.26	205.48	206.96	213.35	211.44	
8	233.51	234.81	236.53	249.94	242.22	
16	274.27	312.74	306.50	325.22	363.80	
32	266.56	335.50	367.32	332.34	373.43	
2	96.63	97.56	89.75	82.47	81.47	FPGA In Nano sec
4	94.72	82.33	81.31	86.62	79.32	
8	81.84	80.70	82.97	79.41	80.25	
16	85.71	84.35	84.43	85.84	84.82	
32	85.92	88.92	90.41	87.88	89.14	

Speed Up Factor Over CPU				
2.25	2.06	2.41	2.50	2.68
2.30	2.50	2.55	2.46	2.67
2.85	2.91	2.85	3.15	3.02
3.20	3.71	3.63	3.79	4.29
3.10	3.77	4.06	3.78	4.19

- Depth: ex. /a/b/c, /a=1, /a/b=2, /a/b/c=3
- Bits: LSBs required for lookup
 - ex. 2^{16} entries need last 16 bits:
...01001011111011110011

Future Work

- **Reduce large size batching**
 - **Originally required to reduce communication overhead**
 - **Not feasible in an online environment as the latency will be high**
- **Reduce extra data copy by providing a shared memory space FPGA can access**
- **Include Longest Prefix Matching as part of the design**
- **Support dispatch for token based input**
- **Integration into the NDN repository**